PL-LVI: A LiDAR-Visual-Inertial SLAM System Integrating Visual Point-Line Features

Feng Hui, Zhiyu Zhou, *Student Member, IEEE*, and Yu Liu, *Senior Member, IEEE*

***Abstract*—Line features provide more structural information than point features, particularly in low-texture, man-made environments. In this paper, we present PL-LVI, a monocular LiDAR-visual-inertial simultaneous localization and mapping (SLAM) system that tightly integrates LiDAR point features with visual line features across all stages of the process. Line features are initially detected by the visual-inertial odometry (VIO), with depth information derived from LiDAR. When feature association fails, triangulation is employed to estimate the missing depth. We then model the point-line reprojection error and optimize the VIO output by minimizing residuals, which serve as the initial guess for LiDAR-inertial odometry (LIO) point cloud registration. Furthermore, we introduce a novel global descriptor that combines the power of point and line features for place recognition, which enhances loop closure detection. Our method is thoroughly evaluated on multimodal datasets, encompassing both indoor and outdoor scenarios, and compared with state-of-the-art techniques. The results demonstrate that PL-LVI significantly improves localization accuracy, global consistency, and robustness in complex environments.**

***Note to Practitioners*—This paper was motivated by the practical need for robust and accurate localization and mapping in environments where single sensor methods struggle, such as in low-light indoor spaces or dynamic urban settings. We introduce PL-LVI, a LiDAR-visual-inertial simultaneous localization and mapping (SLAM) system that fuses visual point and line features with LiDAR depth data. By leveraging the complementary strengths of these sensors, PL-LVI delivers enhanced positioning accuracy, improved map consistency, and real-time performance. Its novel global descriptor facilitates efficient loop closure detection, which is critical for reliable navigation in autonomous driving and indoor robotics. The system is readily adaptable to existing automation platforms, offering practitioners a powerful tool to boost operational efficiency and productivity in challenging real-world scenarios.**

***Index Terms*—Multi-sensor fusion, loop closure detection, point and line fusion, simultaneous localization and mapping (SLAM).**

Received 2 October 2024; revised 31 December 2024 and 5 March 2025; accepted 16 March 2025. Date of publication 10 April 2025; date of current version 11 February 2026. This article was recommended for publication by Editor H. Liu upon evaluation of the reviewers' comments. This work was supported in part by the Key Research and Development Program of Guangdong Province under Grant 2021B0101200001 and in part by the Key Program of National Natural Science Foundation of China under Grant 62433011. *(Corresponding author: Yu Liu.)*

The authors are with the School of Automation Science and Engineering and the Engineering Research Center of Precision Electronic Manufacturing Equipment of Ministry of Education, South China University of Technology, Guangzhou 510640, China (e-mail: auhuifeng@mail.scut.edu.cn; auzyzhou@mail.scut.edu.cn; auylau@scut.edu.cn).

Digital Object Identifier 10.1109/TASE.2025.3559668

I. Introduction

EARLY simultaneous localization and mapping (SLAM) systems [1], [2], [3] primarily relied on a single sensor, either a camera or LiDAR, for localization and mapping. While single sensors offer advantages in terms of weight and cost and can provide relatively accurate localization and mapping results in most environments, they suffer from data loss and instability due to their respective inherent limitations [4]. SLAM technology, particularly in unmanned driving systems, necessitates all-weather capability, making it challenging to meet both localization and mapping requirements solely relying on a single sensor.

Localization and mapping strategies based on multi-sensor fusion [5], [6], [7] have gained significant attention due to their versatility, high accuracy, and robustness. By combining the strengths of multiple sensors, multi-sensor SLAM systems offer broader applicability compared to those relying on a single sensor [8]. This is especially advantageous in environments rich in linear features. Compared to point features, linear features not only span a broader spatial range but also provide more detailed structural information about the environment [9]. These features are particularly beneficial in man-made environments [10], such as outdoor tunnel settings and indoor spaces, where linear features are more prominent. Consequently, when point features are sparse or insufficient in a scene, linear features offer a viable alternative for effective localization and mapping.

In recent years, researchers [11], [12], [13] have made significant advancements in pose estimation within SLAM systems by capitalizing on line features extracted using the Line Segment Detector (LSD) algorithm [14], which exhibit superior geometric properties. The LSD algorithm, originally designed for representing scene structures and shapes, lends itself well to scene recognition tasks within SLAM systems, as depicted in Fig. 1. However, it has been observed that there is limited exploration of line features for loop closure detection. Li et al. [15] retrieve loop closure frames corresponding to the current keyframe from a maintained queue of keyframes to establish correspondence between visual point and line features. Nevertheless, their approach prioritizes point features when computing the pose transformation between two loop closure frames, turning to line features only when an adequate number of point features cannot be matched. Consequently, their approach does not fully integrate line features for loop closure detection.

1545-5955 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

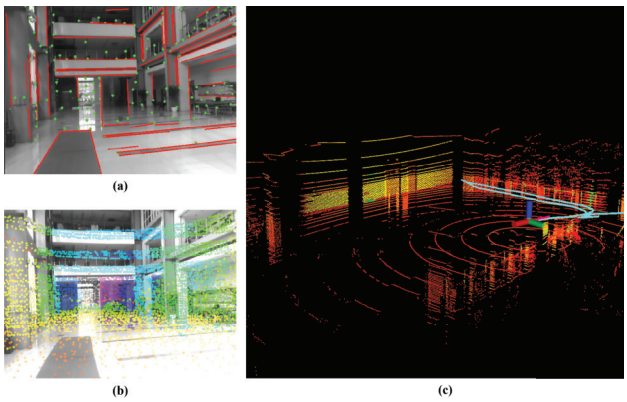


Fig. 1. Multi-sensor fusion performance of PL-LVI in structured indoor environments. (a) Feature extraction results from monocular vision, showing detected point features (green) and line features (red). (b) LiDAR-camera fused depth estimation through projection of LiDAR point clouds onto the image plane, color-coded by depth values. (c) Real-time mapping with LiDAR point cloud reconstruction and robot trajectory visualization (blue).

In this paper, we present PL-LVI, a LiDAR-visual-inertial SLAM system that integrates point and line features for real-time state estimation and map construction. The system consists of two subsystems: a visual-inertial odometry (VIO) module and a LiDAR-inertial odometry (LIO) module. For each frame captured by the monocular camera, the system detects, tracks, and optimizes both point and line features. To estimate the depth of visual line features, we utilize the projection of the LiDAR point cloud to obtain the spatial depth of line segments. If depth estimation fails, triangulation is employed as an alternative. Finally, visual line features and LiDAR point features are fused and incorporated into the loop closure constraints of PL-LVI. The key contributions of this work can be summarized as follows:

1) Implementation of a multi-sensor SLAM system incorporating point and line features. Within the VIO subsystem, spatial coordinates of visual line features are derived through the projection of LiDAR point clouds, while maintaining visual point-line reprojection error and inertial measurement unit (IMU) residuals within a sliding window to ensure precise state estimation.
2) Introduction of a novel global spatial descriptor for loop closure detection, achieved by fusing visual line features and LiDAR point cloud features. This descriptor, characterized by rotation invariance, substantially mitigates the system's cumulative error, thereby enabling globally consistent trajectory estimation.
3) Comprehensive validation of our method using publicly available datasets encompassing diverse and complex scenes.

The subsequent sections of this paper are structured as follows: Section II conducts a review of pertinent literature. Section III furnishes a detailed exposition of our system's components. In Section IV, we delineate the association method between visual features and LiDAR point cloud data, along with the VIO information derived from the fusion of IMU residuals and point-line reprojection errors. Section V provides a comprehensive introduction to our proposed loop closure detection algorithm, which amalgamates visual line features and LiDAR point cloud features to achieve global optimization based on loop closure information. Section VI presents the experimental results and analysis of our system across various complex scenes. Finally, Section VII draws conclusions from our findings and discusses avenues for future research.

II. Related Work

In this section, we will delve into the pertinent literature concerning our system or method, specifically focusing on the advancement of LiDAR-visual-inertial SLAM and visual point-line SLAM. Given the burgeoning diversity of SLAM solutions in recent years, we will judiciously present seminal and most pertinent works within these branches.

A. LiDAR-Visual-Inertial SLAM

In recent years, the refinement of LiDAR SLAM and visual SLAM theories has been notable. With the advancement of these technologies, multi-sensor fusion SLAM has garnered increasing interest, with fusion methodologies primarily categorized into two main types: loosely coupled and tightly coupled fusion approaches.

Loosely coupled methods are recognized for their simplicity, scalability, and lower computational demands. Notably, VIL-SLAM [16] combines stereo VIO with LiDAR mapping to construct dense point cloud maps. The addition of visual loop closures enhances global map consistency. Similar work by Wang et al. [17] extends these concepts, incorporating considerations for fault modules. Zhang and Singh [18] enhance system robustness by using VIO information as prior constraints in LiDAR scan matching. Despite these advancements, the lack of joint optimization of LiDAR data with visual-inertial measurements means these systems remain fundamentally loosely coupled.

On the other hand, tightly coupled methods demonstrate superior performance in terms of accuracy and robustness. For example, LIC-Fusion [5] integrates IMU data with LiDAR edge features and sparse visual features within an MSCKF [19] framework, yielding favorable outcomes. Subsequent work such as LIC-Fusion 2.0 [20] integrates a sliding window approach for planar features and leverages IMU measurements for distortion correction of raw point clouds, resulting in a more robust system. LVI-SAM [6] merges LIO-SAM [21] and VINS-Mono [22] to form a LiDAR-visual-inertial SLAM system on a factor graph optimization framework. This system, composed of two subsystems (LIO and VIO), can independently operate when one subsystem fails or collaboratively when a sufficient number of features are detected. R3LIVE++ [7] focuses on mapping, where the two subsystems maintain independent maps. The LIO subsystem reconstructs the map's geometric structure, while the VIO subsystem restores color, brightness, and other map attributes.

B. Point-Line SLAM

Compared to point features, line features offer greater advantages in providing structural information of real environments, especially in artificial settings. Many classical SLAM

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

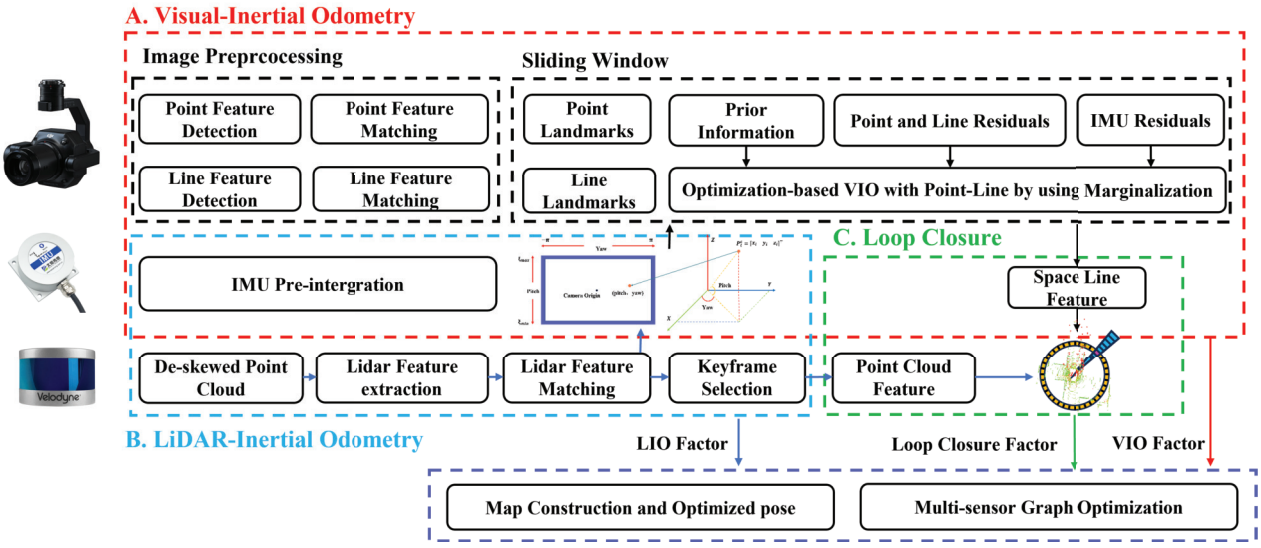


Fig. 2. The diagram delineates the comprehensive pipeline of the proposed PL-LVI system, encompassing three distinct threads: the LiDAR-Inertial Odometry subsystem, the Visual-Inertial Odometry subsystem, and the loop closure detection subsystem.

systems have been extended to integrate point-line feature fusion versions. For instance, PL-SLAM [23] has integrated line features into the classical ORB-SLAM [2] framework, resulting in more robust localization results. Similarly, PL-VIO [11] tightly couples point-line features within the VINS-Mono benchmark framework, albeit without real-time capability. To address this issue, PL-VINS [12] has optimized the front-end line feature extraction algorithm, thus developing the first real-time monocular VINS method based on point-line features. Furthermore, PLS-VINS [13] has proposed a novel line feature error reprojection model and introduced structural constraints of artificial buildings in the backend optimization. Additionally, PLJ-SLAM [24] incorporates coplanarity of line segment pairs based on ORB-SLAM2 [25], introducing distance constraints of spatial lines from multiple views to the connecting points of coplanar lines, thereby enhancing system localization accuracy. UL-SLAM [26], built on ORB-SLAM3 [27], improves robustness and accuracy in complex environments by unifying structural and non-structural constraints and integrating point, structural line, and non-structural line features. PL-EVIO [28], based on the work in [29], uses line features derived from event cameras to provide additional structural and constraint information in artificial scenes, achieving robust and real-time state estimation.

III. System Overview

This paper presents a LiDAR-visual-inertial SLAM method, PL-LVI, which integrates point and line features. The system pipeline is illustrated in Fig. 2. In the following sections, we provide a brief overview of the main modules and components of the system.

A. VIO Subsystem

The detection and tracking of point and line features are performed simultaneously on each image frame captured by the camera. Point features are detected using the Harris corner detection algorithm [30] and subsequently tracked with the sparse optical flow method [31]. Line features are extracted using an enhanced LSD algorithm [14], described with the Line Band Descriptor [32], and matched via the K-Nearest Neighbors algorithm [33]. To integrate LiDAR data, timestamps are used to project the LiDAR point cloud into the camera coordinate system, generating a depth map while filtering out points outside the camera's field of view to obtain depth information for line feature endpoints. For line features lacking depth information, depth estimation is performed via triangulation after image preprocessing. Finally, a joint optimization is conducted by minimizing the visual reprojection error of point and line features along with the residuals from the IMU, ensuring accurate VIO state estimation.

B. LIO Subsystem

Upon the arrival of a new LiDAR scan frame, feature extraction based on local curvature [3] is first conducted on the point cloud to obtain edge and planar features. If the robot's pose change exceeds 1 meter and 10 degrees, a new LiDAR frame is selected as a keyframe. The motion of the robot during LiDAR scanning between two keyframes is estimated using IMU measurements. Subsequently, the current keyframe is matched with the global map [21], and the LiDAR frames between two keyframes are marginalized and used to update the robot's current state in the factor graph.

C. Loop Closure Detection

At the data level, we fuse visual information from VIO and point cloud information from LIO to construct a global spatial descriptor. This descriptor, independent of odometry information, retains the geometric properties of the three-dimensional spatial structure described by visual line features and LiDAR point cloud features, while also possessing rotation invariance. Consequently, it can be effectively utilized for location recognition.

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

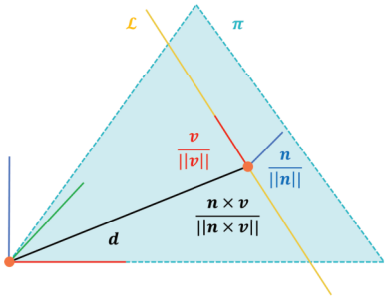


Fig. 3. Transformation of visual line features from Cartesian coordinates to Plücker coordinates.

IV. Point and Line Feature Fusion

Most SLAM systems are traditionally designed to leverage point features for scene recognition and pose estimation. However, in the vast majority of scenarios, line features exhibit significant advantages in terms of stability, geometric information, and motion estimation. Various methods have been proposed to represent spatial lines, with [34] providing a systematic summary of these methods and analyzing their strengths and weaknesses in the optimization and projection transformation processes. We underscore the approach for acquiring depth information of line feature endpoints, a departure from previous works.

A. The Representation of Spatial Lines

1) Plücker Coordinates: Plücker coordinates are commonly employed for the transformation and projection of lines, as depicted in Fig. 3. Assuming a line $\boldsymbol{L} = \left[A, B\right]^\top$ is specified in three-dimensional space by the homogeneous coordinates of its two endpoints, where $A = \left[x_a, y_a, z_a, w_a\right]^\top = \left[\boldsymbol{a}, w_a\right]^\top$ and $B = \left[x_b, y_b, z_b, w_b\right]^\top = \left[\boldsymbol{b}, w_b\right]^\top$, with w representing the inverse depth of the endpoints. Then the line $\mathcal{L}$ in the Plücker coordinate system can be expressed as:

$$\mathcal{L} = \begin{bmatrix} \mathbf{n} \\ \mathbf{v} \end{bmatrix} = \begin{bmatrix} \frac{\boldsymbol{a}}{w_a} - \frac{\boldsymbol{b}}{w_b} \\ \boldsymbol{a} \times \boldsymbol{b} \end{bmatrix} \in \mathbb{R}^6 \tag{1}$$

where $\mathbf{n} \in \mathbb{R}^3$ represents the normal vector to the plane containing vectors $\boldsymbol{a}$ and $\boldsymbol{b}$, and $\mathbf{v} \in \mathbb{R}^3$ represents the direction vector of the line segment.

Lines expressed in Plücker coordinates exhibit the constraint $\mathbf{n}^\top \mathbf{v} = 0$, thereby enabling unique determination of a line within three-dimensional space. However, while the degree of freedom for a line in three-dimensional space is four, the Plücker coordinate system employs six degrees of freedom to represent lines, resulting in a non-compact representation that introduces redundancy. Moreover, over-parameterization can induce instability across the entire system.

2) Orthogonal Coordinates: In fact, during the optimization process, the orthogonal representation with four parameters is commonly used to represent lines. The orthogonal $(\mathbf{U}, \mathbf{W})$ representation of the Plücker coordinates of a straight line can be obtained through QR decomposition:

$$\begin{bmatrix} \mathbf{n} & \mathbf{v} \end{bmatrix} = \begin{bmatrix} \frac{\mathbf{n}}{||\mathbf{n}||} & \frac{\mathbf{v}}{||\mathbf{v}||} & \frac{\mathbf{n} \times \mathbf{v}}{||\mathbf{n} \times \mathbf{v}||} \end{bmatrix} \begin{bmatrix} ||\mathbf{n}|| & 0 \\ 0 & ||\mathbf{v}|| \\ 0 & 0 \end{bmatrix} \tag{2}$$

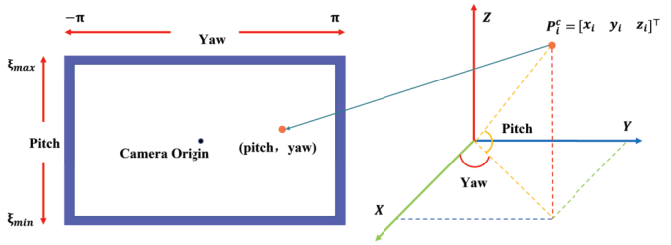


Fig. 4. Projection of LiDAR point cloud onto a plane to form a depth image.

$$\mathbf{U} = \boldsymbol{R}(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\mathbf{n}}{||\mathbf{n}||} & \frac{\mathbf{v}}{||\mathbf{v}||} & \frac{\mathbf{n} \times \mathbf{v}}{||\mathbf{n} \times \mathbf{v}||} \end{bmatrix} \tag{3}$$

$$\mathbf{W} = R(\theta) = \begin{bmatrix} w_1 & -w_2 \\ w_2 & w_1 \end{bmatrix}, \begin{cases} w_1 = \frac{||\mathbf{n}||}{\sqrt{||\mathbf{n}||^2 + ||\mathbf{v}||^2}} \\ w_2 = \frac{||\mathbf{v}||}{\sqrt{||\mathbf{n}||^2 + ||\mathbf{v}||^2}} \end{cases} \tag{4}$$

where $\mathbf{U} \in \mathrm{SO}(3)$ and $\mathbf{W} \in \mathrm{SO}(2)$ stand for the respective matrices of rotation in three and two dimensions. $\boldsymbol{\theta}$ and θ respectively represent three-dimensional vectors and scalars, we can define the standard orthogonal representation of a line with a four-parameter vector Θ:

$$\Theta^\top = \left(\boldsymbol{\theta}^\top, \theta\right) \tag{5}$$

Once a 3D line is represented orthogonally, its Plücker coordinates $\mathcal{L}$ and orthogonal coordinate representation $(\mathbf{U}, \mathbf{W})$ will be transformed following the method outlined below:

$$\mathcal{L} = \left[w_1 \boldsymbol{u}_1^\top, w_2 \boldsymbol{u}_2^\top\right]^\top \tag{6}$$

where $\boldsymbol{u}_i$ represents the i-th column of $\mathbf{U}$, and w_1 and w_2 can be obtained from Eq. (4).

B. Depth Extraction of Line Features

The straight-line reprojection error is defined based on the distance between the observed line endpoints and the reprojected line [9]. The residual magnitude depends on the accuracy of the line endpoints. However, spatial line features may appear with varying lengths on the camera image plane when viewed from different perspectives, leading to significant shifts in endpoint positions and consequently affecting residual values. To address this issue, we utilize depth information from LiDAR. By integrating endpoint data with depth maps generated from LiDAR point clouds, we achieve precise endpoint localization for line segments. Only endpoints without associated depth information undergo triangulation for depth estimation.

1) LiDAR Depth Projection: In the case of a multi-line LiDAR, let us designate the maximum field of view formed by the first and last LiDAR as ξ_{max} and ξ_{min}, respectively. Within a frame of point cloud data, the geometric configuration resulting from the 360-degree rotation of the LiDAR can be envisaged as a hollow cylinder centered around the LiDAR. Projecting this hollow cylinder along the direction perpendicular to its main axis produces a two-dimensional plane. To simplify the process of three-dimensional point cloud plane projection, we choose to employ a single point cloud.

Assuming that after preprocessing, the point cloud P_i^c from the i-th LiDAR is already positioned in the camera coordinate

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

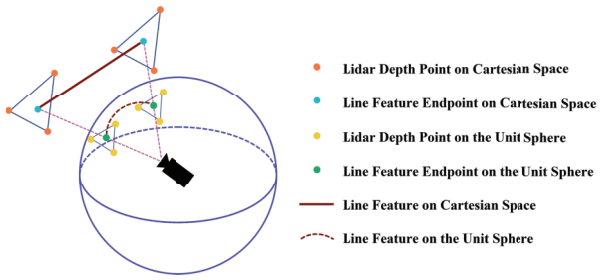


Fig. 5. Depth association between visual line features and LiDAR point cloud.

system c. The angle formed between the line connecting the origin of the coordinate system and the $X-Y$ plane is referred to as the pitch angle, while the angle formed between the line connecting the point and the origin of the coordinate system, projected onto the $X-Z$ plane, is known as the yaw angle. Furthermore, we discretize the pitch angle in the three-dimensional space into row indices represented as ϑ_p, and the yaw angle into column indices represented as ϑ_y. As depicted in Fig. 4, this process leads to the creation of a depth map:

$$\vartheta_y = \tan^{-1}\frac{y_i}{x_i}, \vartheta_p = \sin^{-1}\frac{z_i}{\sqrt{x_i^2+y_i^2+z_i^2}} \tag{7}$$

Subsequently, the depth map is downsampled to maintain uniform density and stored in a KD-Tree structure. Following this, a method similar to the one described in [18] is employed. The LiDAR point cloud extracted from the current depth map and the line features detected by the camera are simultaneously projected onto a unit sphere centered at the camera, as shown in Fig. 5. On this unit sphere, the LiDAR point cloud closest to the endpoints of the three nearest spatial line features is then identified. However, occlusions often occur between consecutive camera frames, leading to significant inaccuracies in depth estimation. To ensure the validity of the depth association, the spatial distance between the three points is computed. Specifically, if the maximum distance between the three nearest point clouds exceeds 3 meters, it is concluded that these point clouds belong to different objects, indicating a failure in depth association. In contrast, if the distance is less than 3 meters, the depth association is considered successful, and the depth of the endpoints is attributed to the geometric center of the triangle formed by the three point clouds. For line features where depth retrieval is challenging, the depth of spatial line segments is obtained through triangulation of the line segments.

2) Triangulation of Spatial Line Segments: Based on the methodology described in [35], the initial procedure entails the transformation of coordinates for the straight lines.

$$\mathcal{L}_c = \begin{bmatrix}\mathbf{n}_c\\ \mathbf{v}_c\end{bmatrix} = \boldsymbol{\mathcal{H}}_{cw}\mathcal{L}_w = \begin{bmatrix}\boldsymbol{R}_{cw} & [\boldsymbol{t}_{cw}]_\times \boldsymbol{R}_{cw}\\ \mathbf{0} & \boldsymbol{R}_{cw}\end{bmatrix} \tag{8}$$

where $[\cdot]_\times$ denotes the skew-symmetric matrix of a vector, $\boldsymbol{\mathcal{H}}_{cw}$, $\boldsymbol{R}_{cw}$, and $\boldsymbol{t}_{cw}$ respectively represent the transformation matrix, rotation matrix, and translation vector used to project a line from the world coordinate system to the camera coordinate system.

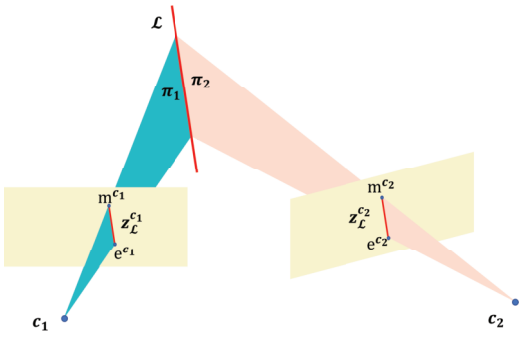


Fig. 6. Initialization failure of depth association for visual line features.

When observing the same landmark in different camera frames, we can easily compute the Plücker coordinates of the landmark. As illustrated in Fig. 6, let's assume the observation of the same spatial line $\mathcal{L}_c$ at moments c_1 and c_2, with its projection onto the camera's normalized plane being $\mathbf{z}_{\mathcal{L}}^{c_1}$ and $\mathbf{z}_{\mathcal{L}}^{c_2}$ respectively. The projected line segment $\mathbf{z}_{\mathcal{L}}^{c_1}$ can be represented by two endpoints $\mathbf{m}^{c_1} = [u_m, v_m, 1]^\top$ and $\mathbf{e}^{c_1} = [u_e, v_e, 1]^\top$. By adding the non-collinear coordinate origin $O = [x_o, y_o, z_o]^\top$, the line segment $\mathbf{z}_{\mathcal{L}}^{c_1}$ uniquely determines a plane $\boldsymbol{\pi} = [\pi_x, \pi_y, \pi_z, \pi_w]^\top$ in three-dimensional space satisfying:

$$\begin{cases}\pi_x(x-x_o)+\pi_y(y-y_o)+\pi_z(z-z_o)=0\\ [\pi_x,\pi_y,\pi_z]^\top = \left[\mathbf{m}^{c_1}\right]_\times \mathbf{e}^{c_1}\\ \pi_w = \pi_x x_o + \pi_y y_o + \pi_z z_o\end{cases} \tag{9}$$

Through the planes $\boldsymbol{\pi}_1$ and $\boldsymbol{\pi}_2$ of the camera's normalized coordinate system, we can obtain the dual Plücker matrix $\mathcal{L}^*$ of the spatial line as follows:

$$\mathcal{L}^* = \begin{bmatrix}[\mathbf{v}]_\times & \mathbf{n}\\ -\mathbf{n}^\top & 0\end{bmatrix} = \boldsymbol{\pi}_1\boldsymbol{\pi}_2^\top - \boldsymbol{\pi}_2^\top\boldsymbol{\pi}_1 \tag{10}$$

The above equation readily allows for the solution of the Plücker coordinates $\mathcal{L}_c$ in camera frame c_1 from the dual Plücker matrix $\mathcal{L}^*$.

C. Reprojection Error Model for Visual Features

Following the detection, tracking, matching, and triangulation processes, visual features require further re-projection error minimization to determine camera pose and constrain the biases and noise of the IMU. Assuming observation of the α-th visual point feature $\mathbf{z}_{\mathbf{f}_\alpha}^{c_\gamma} = [u_{\mathbf{f}_\alpha}^{c_\gamma}, v_{\mathbf{f}_\alpha}^{c_\gamma}, 1]^\top$, in the γ-th camera frame, the re-projection error $\mathbf{r}_f\left(\mathbf{z}_{\mathbf{f}_\alpha}^{c_\gamma}, \mathcal{X}\right)$, of a three-dimensional spatial point, as described in [22], is defined as the distance between the projected point and the observed point $\mathbf{f}_\alpha^{c_\gamma}$:

$$\begin{cases}\mathbf{r}_f\left(\mathbf{z}_{\mathbf{f}_\alpha}^{c_\gamma}, \mathcal{X}\right) = \begin{bmatrix}\frac{x^{c_\gamma}}{z^{c_\gamma}} - u_{\mathbf{f}_\alpha}^{c_\gamma}\\ \frac{y^{c_\gamma}}{z^{c_\gamma}} - v_{\mathbf{f}_\alpha}^{c_\gamma}\end{bmatrix}\\ \mathbf{f}_\alpha^{c_\gamma} = \left[x^{c_\gamma}, y^{c_\gamma}, z^{c_\gamma}\right]^\top\end{cases} \tag{11}$$

where $\mathcal{X}$ represents the state vector. Similarly, the re-projection error $\mathbf{r}_l\left(\mathbf{z}_{\mathcal{L}_\beta}^{c_\gamma}, \mathcal{X}\right)$, for line features is defined as the

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

distance between the endpoints of the line segment and the projected line:

$$\begin{cases} \mathbf{r}_l\left(\mathbf{z}_{\mathcal{L}_\beta}^{c_\gamma}, \mathcal{X}\right) = \begin{bmatrix} d\left(\mathbf{m}_\beta^{c_\gamma}, F_\beta^{c_\gamma}\right) \\ d\left(\mathbf{e}_\beta^{c_\gamma}, F_\beta^{c_\gamma}\right) X \end{bmatrix} \\ d\left(\mathbf{m}_\beta^{c_\gamma}, F_\beta^{c_\gamma}\right) = \dfrac{\mathbf{m}_\beta^{c_\gamma\top} F_\beta^{c_\gamma}}{\sqrt{l_1^2 + l_2^2}} \\ F_\beta^{c_\gamma} = [l_1, l_2, l_3]^\top = \mathcal{K}\mathbf{n}_\beta \end{cases} \tag{12}$$

where $d(\cdot)$ denotes the distance function from a point to a line, while $\mathbf{m}_\beta^{c_\gamma}$ and $\mathbf{e}_\beta^{c_\gamma}$ represent the homogeneous coordinates of the endpoints of the β-th spatial line feature observed in the γ-th camera coordinate system. $F_\beta^{c_\gamma}$ denotes the projection line obtained by projecting the β-th spatial line onto the camera plane using the projection matrix $\mathcal{K}$ within the same camera frame, and $[l_1, l_2, l_3]^\top$ represents the coordinates after projection.

D. Sliding Window Optimization

The IMU pre-integration residuals and the point-line re-projection errors are maintained within a fixed sliding window for real-time optimization. We define the complete state vector $\mathcal{X}$ at time h, along with the IMU coordinate system's position $\mathbf{p}_h^w$, orientation $\mathbf{q}_h^w$, velocity $\mathbf{v}_h^w$, accelerometer bias $\mathbf{b}_a$, and gyroscope bias $\mathbf{b}_g$ in the world coordinate system w as the set $\mathbf{x}_h$:

$$\begin{cases} \mathcal{X} = \left[\mathbf{x}_n, \cdots, \mathbf{x}_{n+N}, \lambda_m, \cdots, \lambda_{m+M}, \mathcal{O}_g, \cdots, \mathcal{O}_{g+G}\right]^\top \\ \mathbf{x}_h = \left[\mathbf{p}_h^w, \mathbf{q}_h^w, \mathbf{v}_h^w, \mathbf{b}_a, \mathbf{b}_g\right]^\top, h \in [n, n+N] \end{cases} \tag{13}$$

where the subscripts n, m and g respectively represent the starting indices for the IMU state, visual point feature landmarks, and visual line feature landmarks. N denotes the number of keyframes in the sliding window, while M and G represent the quantities of observed point features and line features across all keyframes within the sliding window. λ_m represents the inverse depth, serving as the parameterization for the m-th point landmark observed in the first keyframe. $\mathcal{O}_g$ denotes the orthogonal representation of the g-th line feature in the world coordinate system. Further optimization is carried out by minimizing all residuals $\mathbf{C}$ to refine all state vectors within the sliding window:

$$\mathbf{C} = \min_{\mathcal{X}} \left\{ e_{prior} + e_{imu} + e_{point} + e_{line} \right\} \tag{14}$$

where e_{prior} and e_{imu} respectively denote the prior information of the old frame and the IMU pre-integration residuals, and e_{point} and e_{line} represent the sets of re-projection error terms for visual point features and visual line features:

$$\begin{cases} e_{porior} = \rho\left(\|\mathbf{r}_p - \mathrm{J}_p\mathcal{X}\|_{\sum_p}^2\right) \\ e_{imu} = \sum\limits_{\gamma\in\mathcal{B}} \rho\left(\|\mathbf{r}_b\left(\mathbf{z}_{b_\gamma b_{\gamma+1}}, \mathcal{X}\right)\|_{\sum_{b_\gamma b_{\gamma+1}}}^2\right) \\ e_{point} = \sum\limits_{(\gamma,\alpha)\in\mathbf{F}} \rho\left(\|\mathbf{r}_f\left(\mathbf{z}_{\mathbf{F}_\alpha}^{c_\gamma}, \mathcal{X}\right)\|_{\sum_{\mathbf{F}_\alpha}^{c_\gamma}}^2\right) \\ e_{line} = \sum\limits_{(\gamma,\beta)\in\mathbf{L}} \rho\left(\|\mathbf{r}_l\left(\mathbf{z}_{\mathbf{L}_\beta}^{c_\gamma}, \mathcal{X}\right)\|_{\sum_{\mathbf{L}_\beta}^{c_\gamma}}^2\right) \end{cases} \tag{15}$$

where $\mathbf{r}_p$ denotes the prior information calculated from the frames within the marginalized sliding window, and $\mathrm{J_p}$ is the prior Jacobian matrix derived from the Hessian matrix obtained after previous optimization. $\mathbf{r}_b$ represents the residuals from the IMU measurements, and $\mathcal{B}$ is the set of pre-integrated IMU measurements in the sliding window. $(\gamma, \alpha) \in \mathbf{F}$ and $(\gamma, \beta) \in \mathbf{L}$ respectively denote the α-th three-dimensional point $\mathbf{F}_\alpha$ and the β-th spatial line $\mathbf{L}_\beta$ observed in the γ-th camera coordinate system c_γ. $\sum_{\gamma\in\mathcal{B}}$, $\sum_{\mathbf{F}_\alpha}^{c_\gamma}$ and $\sum_{\mathbf{L}_\beta}^{c_\gamma}$ are the covariance matrices for IMU measurements, point feature measurements and line feature measurements. $\rho(\cdot)$ denotes the Huber norm, which is employed to suppress outliers.:

$$\rho(\mu) = \begin{cases} \mu & \mu \leqslant 1 \\ 2\sqrt{\mu} - 1 & \mu > 1 \end{cases} \tag{16}$$

Finally, the Levenberg-Marquardt algorithm is utilized to solve the nonlinear optimization problem. Specifically, the orthogonal representation of line features can be iteratively updated through the joint Eq. (3) and Eq. (4):

$$\begin{cases} \mathbf{U}_t\left(\mathbf{I} + \left[\delta\boldsymbol{\theta}\right]_\times\right) \to \mathbf{U}_{t+1} \\ \mathbf{W}_t\left(\mathbf{I} + \begin{bmatrix} 0 & -\delta\theta \\ \delta\theta & 0 \end{bmatrix}\right) \to \mathbf{W}_{t+1} \end{cases} \tag{17}$$

where $\mathbf{I}$ is the identity matrix.

V. Loop Clousre Detection

To address the challenges associated with long-duration, large-scale scene localization and trajectory consistency in SLAM systems, this paper proposes a loop closure detection algorithm termed Point-Line Context (PLC), which is based on Scan Context (SC) [36]. Unlike conventional SC methods, which solely rely on the height information of points to characterize the distribution patterns of point clouds, PLC not only leverages data from LiDAR-acquired point clouds but also aligns with the original objective of the LSD line segment detection algorithm, aimed at effectively representing scene structures. Consequently, this approach incorporates the geometric information of visual line features, enhancing its applicability in the domain of location recognition.

A. Point-Line Context

First, we represent three-dimensional space as a set of points $\mathbf{P} = \{\mathbf{p}_1, \mathbf{p}_2, \cdots, \mathbf{p}_\delta, \cdots, \mathbf{p}_N\}$ and transform it from a local Cartesian coordinate system to a polar coordinate system containing only $X-Y$ coordinates. Then, we partition three-dimensional space into toroidal subspace N_r and sector subspace N_s with the sensor as the center at equidistant intervals. Any point $\mathbf{p}_\delta$ in space uniquely corresponds to a subspace S_{ij}:

$$S_{ij} = \left\{ \mathbf{p}_\delta \in \mathbf{P} \mid \frac{\gamma_\delta \cdot N_r}{L_{max}} - 1 < i \leq \frac{\gamma_\delta \cdot N_r}{L_{max}}, \frac{(\sigma_\delta + \pi) \cdot N_s}{2\pi} - 1 < j \leq \frac{(\sigma_\delta + \pi) \cdot N_s}{2\pi} \right\} \tag{18}$$

where γ_δ and σ_δ are the axis components of point $\mathbf{p}_\delta$ in polar coordinates, and i and j are the row and column coordinates of

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

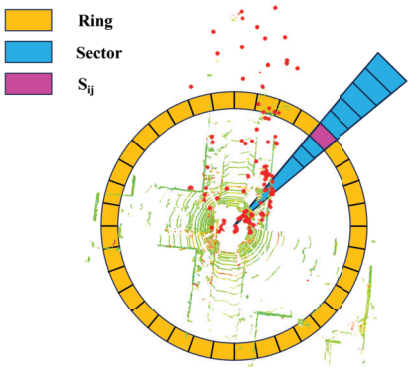


Fig. 7. The visualization example of PLC depicts various components:: LiDAR point cloud features (green dots), midpoints of visual line features (red dots), loop closure candidates (yellow squares), and sector candidates (blue squares).

space, respectively. L_{max} is the maximum distance detectable by the LiDAR.

Before performing feature selection and encoding for each subspace, it is essential to synchronize the data from the LiDAR and the camera. Sensor data synchronization is achieved by stacking the camera data between consecutive LiDAR frames and back-projecting it into the LiDAR coordinate system. Additionally, considering that the length of a spatial line may span multiple subspaces, we represent its association with a specific subspace by using the midpoint $\mathbf{M}^l$ of the line feature, as shown in Fig. 7. Subsequently, in combination with the LiDAR point cloud P^l, we encode the subspace using the encoding function $\varphi(\cdot)$:

$$\varphi\left(S_{ij}\right) = \left[max\ z\left(P^l\right) + max\ f\left(\mathbf{M}^l\right)\right] \in \mathbb{R} \tag{19}$$

where $z(\cdot)$ denotes the z-value of the point cloud returns, and $f(\cdot)$ represents the return length of the spatial line. Since stable line features are typically associated with static objects, we assume that dynamic objects have a height of less than 3 meters. As such, line features with a midpoint height greater than 3 meters are considered stable. After encoding, the global spatial descriptor of the 3D space, denoted as PLC, is defined as an $N_r \times N_s$ matrix $\boldsymbol{\phi}$:

$$\boldsymbol{\phi} = \left[\mathrm{H}_{ij}\right] \in \mathbb{R}^{N_r \times N_s} \tag{20}$$

where $\mathrm{H}_{ij} \in \varphi(S_{ij})$. This indicates that when $S_{ij} \in \varnothing$ is an empty set (no LiDAR scan data and visual line features), $\varphi(S_{ij}) = 0$. This implies that PLC can still function normally when LiDAR or visual cameras degrade.

B. Quick Search

Loop closure detection entails computing the similarity between two locations. Although we have represented three-dimensional space in matrix form, this representation lacks rotational invariance, and matching and searching N-dimensional matrices consumes significant computational resources. To keep search time within reasonable bounds, we propose a rapid search mechanism.

We extract column vectors $\boldsymbol{\eta}$ from matrix $\boldsymbol{\phi}$, where $\boldsymbol{\eta}$ represents the mean of each row in matrix $\boldsymbol{\phi}$:

$$\begin{cases} \boldsymbol{\eta} = \left[\Omega\left(k_1\right), \cdots, \Omega\left(k_r\right)\right]^\top \\ \Omega\left(k_i\right) = \dfrac{\sum_{j=1}^{s} |\mathrm{H}_{ij}|}{N_s} \end{cases} \tag{21}$$

As $\Omega(k_i)$ remains unaffected by viewpoint, vector $\boldsymbol{\eta}$ possesses rotational invariance. We employ $\boldsymbol{\eta}$ to construct a KD-Tree for swiftly exploring potential candidate locations.

C. Offset Matching

While $\boldsymbol{\eta}$ exhibits rotational invariance, adjusting for perspective remains necessary. Considering that the columns of matrix $\boldsymbol{\phi}$ are solely related to sensor installation positions, maintaining consistent row vector sequences, cosine distance can be employed to calculate the similarity between two PLC matrices:

$$\Delta\left(\boldsymbol{\phi}^a, \boldsymbol{\phi}^e\right) = \frac{1}{N_s} \sum_{i=1}^{N_s} \left(\frac{\mathbf{v}_i^a \mathbf{v}_i^e}{\|\mathbf{v}_i^a\| \ \|\mathbf{v}_i^e\|} \right) \tag{22}$$

where $\mathbf{v}_i^a$ and $\mathbf{v}_i^e$ represent the i-th row vectors of matrices $\boldsymbol{\phi}^a$ and $\boldsymbol{\phi}^e$, respectively. To alleviate matching failures due to viewpoint deviations, we minimize the column displacement of $\boldsymbol{\phi}$ to seek the optimal matching position. Defining matrix $\boldsymbol{\phi}_K^e$ as the result of translating matrix $\boldsymbol{\phi}^e$ by K columns, we quantify the optimal column displacement K^* and distance $\Phi(\boldsymbol{\phi}^a, \boldsymbol{\phi}^e)$:

$$\begin{cases} K^* = \underset{K \in [1, N_s]}{\arg\max}\, \Delta\left(\boldsymbol{\phi}^a, \boldsymbol{\phi}_K^e\right) \\ \Phi\left(\boldsymbol{\phi}^a, \boldsymbol{\phi}^e\right) = \underset{K \in [1, N_s]}{\max}\, \Delta\left(\boldsymbol{\phi}^a, \phi_K^e\right) \end{cases} \tag{23}$$

Further, eliminating unsuitable instances of $\Phi(\boldsymbol{\phi}^a, \boldsymbol{\phi}^e)$, the offset information K^* will provide initial values for Iterative Closest Point.

VI. Experiment

To validate the efficacy of the proposed PL-LVI system, this section conducts experimental comparisons primarily between PL-LVI and other state-of-the-art methods, accompanied by corresponding experimental results and analyses to ascertain the system's performance. Evaluations were performed on two publicly available datasets, UrbanNav [37] and M2DGR [38], to assess the system's performance across diverse scenarios.

A. Experiment Setup

This study evaluates the localization accuracy of the proposed system through Absolute Trajectory Error (ATE) analysis and trajectory visualization, and compares it with VINS-Mono [22], LeGO-LOAM [39], LIO-SAM [21], FAST-LIO2 [40], and LVI-SAM [6]. Additionally, in the context of mapping, we assess the loop closure detection strategies based on odometry distance [21], DBoW2 [41], and PLC for their ability to maintain global map consistency over long-term operation. We further conduct a statistical analysis of the precision and recall rates for various loop closure strategies, including SC [36]. All experiments were conducted on a

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE EVALUATION OF ABSOLUTE TRAJECTORY ERROR (ATE, METERS) ON URBANNAV DATASET

Sequence	Length (m)	FAST-LIO2	VINS-Mono		LeGO-LOAM		LIO-SAM		LVI-SAM		PL-LVI	
			(w/o)	(w/)	(w/o)	(w/)	(w/o)	(w/)	(w/o)	(w/)	(w/o)	(w/)
HK-TST	3640	8.625	199.663	180.245	1100.472	1006.831	11.021	11.024	11.092	11.093	10.833	**1.770**
Wham	4860	6.684	356.352	361.551	2841.634	853.691	10.789	9.663	11.168	9.855	10.858	**2.861**
Average	4250	7.655	277.993	270.898	1971.053	930.261	10.905	10.343	11.130	10.474	10.844	**2.32**(↓**78%**)

* FAST-LIO2 implementation excludes loop closure detection capabilities.
* (w/o) and (w/) denote configurations without and with loop closure constraints respectively.
* Bold values indicate the best performance metrics.
* Percentage reduction in parentheses reflects relative improvement over baseline methods.

desktop computer with an Intel i7-10700 CPU, running the Linux-based Robot Operating System (ROS) [42] on Ubuntu 20.04. To ensure a fair comparison, when evaluating the performance of other SLAM systems, necessary hardware adjustments were made to accommodate the datasets, while strictly adhering to the default configurations provided in the respective GitHub repositories. It should be noted that all experiments in this study were performed using the CPU only, without GPU acceleration.

B. UrbanNav Dataset

The UrbanNav dataset offers a comprehensive autonomous driving dataset that captures authentic urban environments within Hong Kong's cityscape. It encompasses diverse scenarios such as high-rise buildings, underground tunnels, narrow streets, and dense pedestrian areas, effectively representing typical urban navigation challenges. Notably, specific sequences contain revisited routes through identical geographical locations, which enables rigorous validation of the loop closure detection algorithm proposed in this study.

1) Evaluation of Localization Accuracy for Different Systems: The dataset, HK-TST and Wham, utilized in this study depict expansive outdoor settings. ATE serves as the metric to assess the deviation between estimated and actual trajectories, as delineated in TABLE I. Notably, PL-LVI yields superior outcomes across both sequences. This outcome can be rationalized by the prevalence of artificial structures within urban landscapes, wherein linear features are more abundant than in natural settings. Specifically, the Wham sequence encompasses protracted, linear tunnel segments, as depicted in Fig. 8. Amidst structured environments and dynamic luminance shifts, visually discernible point features exhibit marked attenuation and instability. Consequently, reliance solely on individual point features within a SLAM system proves inadequate for meeting the localization requisites of intricate urban scenarios. Conversely, line segment features encapsulate crucial environmental structural data, such as lane demarcations and tunnel boundaries, enduringly discernible throughout extended system operation, thus signifying their pivotal role in localization accuracy.

Fig. 9 presents the trajectory maps of both systems. PL-LVI not only achieves the lowest absolute trajectory error but also improves positioning accuracy by 78% compared

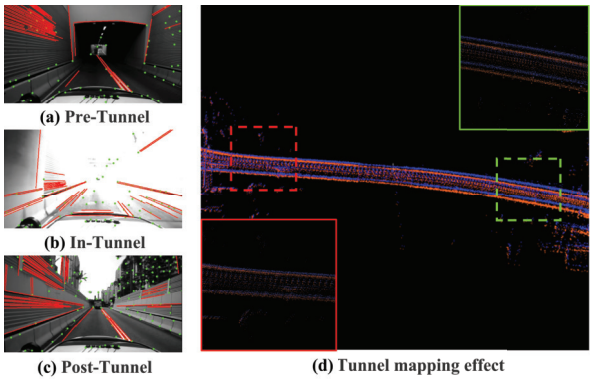


Fig. 8. Illustration from the Wham sequence, in which an autonomous vehicle traverses an urban tunnel. (a)-(c) show the detection of visual point features (green) and line features (red) in pre-tunnel, in-tunnel, and post-tunnel scenarios, respectively. (d) highlights the localized mapping effect within the tunnel by comparing the visual line feature thread when the PL-LVI module is enabled (blue) versus disabled (red).

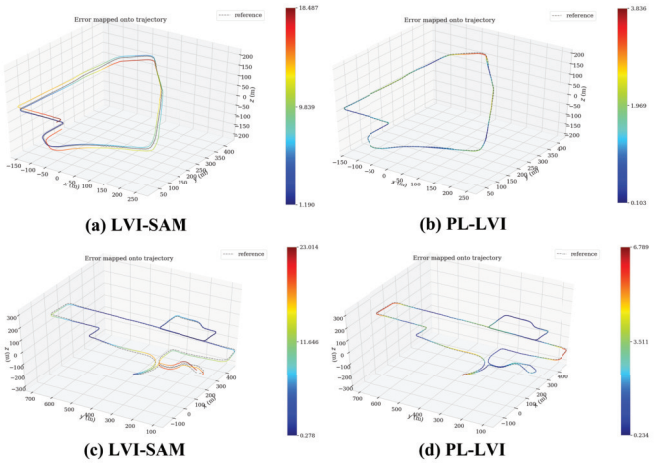


Fig. 9. Comparison of trajectory errors between PL-LVI and LVI-SAM on the UrbanNav dataset. The gray dashed line represents the ground truth trajectory, while the system's estimated trajectory is color-coded based on the corresponding ATE values.

to LVI-SAM. The LVI-SAM trajectory exhibits maximum deviations of 18m and 23m, predominantly along the z-axis. In comparison, the PL-LVI trajectory closely mirrors the ground truth, with maximum deviations of 3.8m and 6.8m,

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

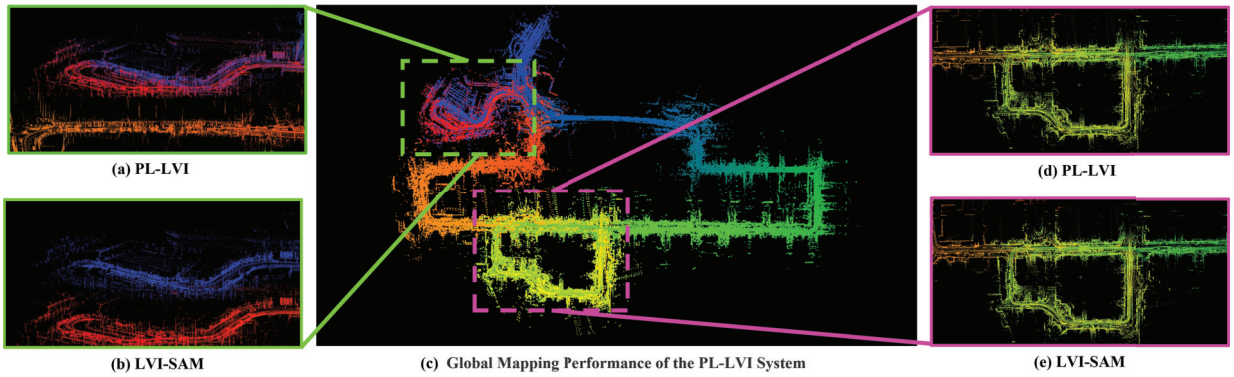


Fig. 10. A comparison of mapping effectiveness between the PL-LVI and LVI-SAM systems on the Wham sequence reveals the dynamic evolution of point cloud coloration over time. (c) illustrates the overall mapping performance of PL-LVI within the Wham sequence, where pink and green rectangles denote the locations of forward and backward loop closures, respectively. (a) presents a locally consistent map generated by PL-LVI following the detection of a backward loop closure, whereas (b) shows a local map produced by LVI-SAM in the absence of such detection. (d) and (e) depict local maps generated by both systems after the successful identification of forward loop closures.

TABLE II
PERFORMANCE METRICS OF DIFFERENT LOOP CLOSURE DETECTION METHODS

Methods	TP	FP	FN	Precision(%)	Recall(%)
DBoW2	34	2	8	89	80.9
Odometry	103	12	24	89.6	81.1
SC	88	4	2	95.7	97.8
PLC	90	2	0	**97.8**	**100**

representing only 0.0824% and 0.1397% of the total travel distance, respectively. This highlights the decisive impact of an effective loop closure detection strategy on positioning accuracy, particularly in large-scale, dynamic urban canyon environments.

2) Performance Evaluation of Loop Closure Detection Strategies: The Wham sequence contains two types of location revisits, one of which is a reverse visit, as shown in Fig. 10. This poses a significant challenge to various loop closure detection strategies. We systematically compare the performance of different loop closure detection methods on this sequence, evaluating the number of detected loops, as well as precision and recall. Since the true loop positions in the UrbanNav dataset are not annotated, we adopt the approach from the literature [43]: if the distance between loop nodes is less than 2 meters, it is considered a true positive; otherwise, it is classified as a false positive. Detailed statistical results are provided in TABLE II.

From a data perspective, detecting forward loop closures (depicted in Fig. 10 (d) and Fig. 10 (e)) is relatively straightforward in loop closure detection tasks. This is primarily attributed to the longer loop paths and higher loop frequencies observed at these localities, providing ample detection time and a higher fault tolerance for the loop closure detection module of LVI-SAM. Even occasional missed detections do not significantly impede overall mapping effectiveness. However, accurately detecting loop closures poses a particular challenge when revisiting previously explored areas from different angles and directions. Fig. 10 (a) and Fig. 10 (b) illustrate reverse loop closure instances in this sequence, where the starting and ending points coincide.

As scene scale and runtime increase, although the loop closure detection algorithm of LVI-SAM can identify loop closure frames and rectify poses at previous forward revisit locations, the accumulation of minor errors over time in large-scale scenes renders Euclidean distance-based loop closure detection methods [21] ineffective. Furthermore, due to reverse site visits, DBoW2 descriptors may lack robust rotational and translational invariance, failing to maintain descriptor content and structure unchanged with significant changes in sensor viewpoint. This leads to substantial disparities in generated descriptors and subsequent failure in loop closure detection. In contrast, PLC relies on distance-independent information and achieves location recognition by retrieving previously stored spatial absolute geometric structures. The descriptors themselves exhibit rotational invariance and mitigate viewpoint deviation effects, thus rectifying their own poses to achieve accurate loop closure registration and ultimately construct globally consistent maps.

C. M2DGR Dataset

M2DGR, compiled by a collegiate robotics team, is a comprehensive dataset spanning diverse scenarios. Key challenges include long corridors, low-light environments, and indoor-outdoor transitions. Although these conditions are common in ground robot applications, conventional SLAM systems often perform suboptimally. Notably, the dataset contains few location revisits across its sequences. As a result, loop closure detection is disabled for all SLAM frameworks, with the evaluation focusing solely on odometry.

TABLE III shows the ATE of various SLAM methods in the indoor scenarios of this dataset. The PL-LVI system achieves an average ATE of 0.382 meters. Compared to the state-of-the-art LVI-SAM, PL-LVI demonstrates a 15% improvement in odometry accuracy and yields the smallest ATE in the majority of indoor sequences. This performance enhancement is primarily attributed to the integration of line features, which

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE EVALUATION OF ABSOLUTE TRAJECTORY ERROR (ATE, METERS) ON M2DGR DATASET

Sequence	Environment	Lighting	Length (m)	Duration (s)	VINS-Mono	LeGO-LOAM	LIO-SAM	LVI-SAM	PL-LVI
room_01	indoor	bright	27.171	72	0.348	**0.147**	0.154	0.153	0.152
room_02	indoor	bright	45.83	75	0.356	0.132	0.124	0.121	**0.119**
room_03	indoor	bright	71.12	128	0.490	0.167	**0.166**	**0.166**	**0.166**
roomdark_01	indoor	dark	38.03	111	-	12.622	**0.155**	0.157	0.156
roomdark_02	indoor	dark	62.80	165	-	12.357	0.437	-	**0.174**
hall_01	indoor	bright	214.03	351	13.337	80.753	0.286	**0.284**	**0.284**
hall_02	indoor	bright	109.46	128	6.834	11.229	0.357	0.300	**0.298**
hall_03	indoor	bright	144.65	164	122.422	28.474	0.527	0.359	**0.355**
hall_04	indoor	bright	91.51	181	3.410	41.301	1.079	**1.077**	**1.077**
hall_05	indoor	bright	285.51	402	5.279	5.082	1.033	1.032	**1.030**
door_01	indoor/outdoor	bright	142.78	461	0.694	-	0.268	**0.261**	0.281
door_02	indoor/outdoor	bright	57.36	127	2.164	0.201	0.250	0.481	**0.193**
circle_01	indoor	bright	316.58	234	3.583	4.611	0.641	0.782	**0.476**
circle_02	indoor	bright	301.45	244	22.171	**0.297**	0.578	0.678	0.590
Average			136.31	203	15.091	15.183	0.433	0.450	**0.382 (↓15%)**

* Loop closure constraints were disabled across all systems in the M2DGR dataset evaluation.
* Bold values denote optimal performance metrics, while dashes (-) indicate system failure or unavailable measurements.
* Percentage reduction in parentheses reflects relative improvement over baseline methods.

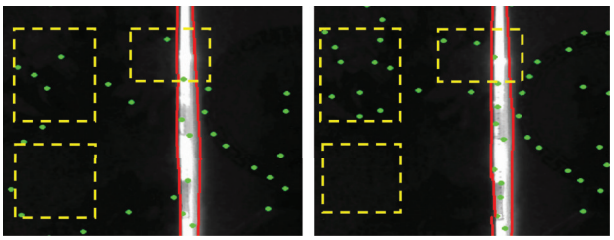


Fig. 11. Tracking and matching results of visual point-line features in dark conditions from the roomdark_01 sequence by PL-LVI. The left image shows disappearing point features (in green within yellow boxes) in the subsequent frame on the right.

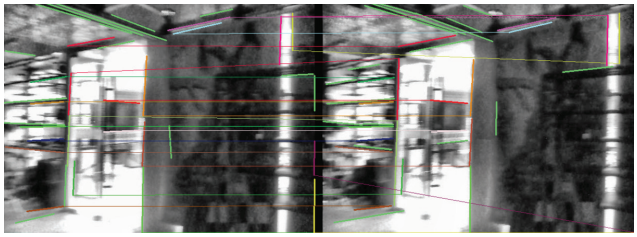


Fig. 12. Tracking and matching results of line features between two frames in the roomdark_02 sequence by PL-LVI, which involves significant lighting changes and pure rotational motion.

provide stronger constraints, thereby improving the robustness and versatility of the PL-LVI system.

In low-light environments, detecting visual point features is particularly challenging [44], often causing significant pose estimation errors. In the roomdark_01 sequence, VINS-Mono fails to operate reliably, likely due to its inability to extract and match features under poor lighting, ultimately leading to pose estimation failure. Among the evaluated systems, LIO-SAM exhibits the most robust performance, primarily because it operates independently of visual data. In contrast, LVI-SAM, which integrates both VIO and LIO subsystems, performs worse than LIO-SAM in this sequence. This degradation stems from the failure of the VIO subsystem, which introduces erroneous bias that negatively impacts LIO, leading to suboptimal trajectory estimation. Nevertheless, the estimated trajectory in PL-LVI benefits from incorporating line features, which enhance the robustness and stability of the VIO subsystem. As shown in Fig. 11, in low-light conditions, monocular cameras exhibit significantly higher correct matching and continuous tracking rates for line features than point features. Consequently, the trajectory accuracy obtained using line features surpasses that of LVI-SAM relying solely on visual points, approaching LIO-SAM's accuracy.

In the roomdark_02 sequence, even the LVI-SAM, which includes a visual-inertial subsystem, fails to function properly under these conditions. This may be attributed to the fact that, compared to the previous sequence, this sequence not only experiences low lighting environments but also accompanies drastic lighting changes and pure rotations, as illustrated in Fig. 12. Visual line features exhibit distinct geometric structures and maintain strong continuity between consecutive image frames during pure rotational motion, even in dimly lit environments. The system proposed in this paper is capable of correctly matching line features in such environments, thereby significantly reducing the impact of lighting changes on the VIO subsystem. In fact, in daily life, drastic lighting changes often accompany transitions between indoor and outdoor environments, which are quite common for robots to navigate

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
AVERAGE TIME CONSUMPTION (MILLISECONDS) OF INDIVIDUAL MODULES IN LVI-SAM AND PL-LVI ON THE URBANNAV DATASET

Strategies	LVI-SAM	PL-LVI
LIO	7.41	7.44
VIO	24.33	57.87
Optimization	98.71	102.96

between. Therefore, the introduction of line features provides a beneficial reference direction for addressing the stability issues of SLAM systems under scenarios of drastic lighting changes.

D. Real-Time Analysis

A module-level real-time analysis was conducted on the UrbanNav dataset to quantitatively evaluate the computational efficiency of PL-LVI and LVI-SAM, with the results presented in TABLE IV. The LIO module in PL-LVI exhibits computational efficiency comparable to that of LVI-SAM, indicating that the incorporation of visual line features introduces minimal additional overhead to the LiDAR processing pipeline. While the inclusion of line features increases the computational load of the VIO module, this addition increases the latency of the joint optimization module by only 4.3%, without significantly increasing the complexity of nonlinear optimization. Nevertheless, this additional computational cost in the VIO module is justified, as it enhances environmental representation while maintaining real-time localization and mapping capabilities on a desktop computer.

VII. CONCLUSION

This paper introduces PL-LVI, a LiDAR-visual-inertial multi-modal SLAM system that significantly enhances localization accuracy and global consistency in complex environments by integrating point-line features with multi-sensor data. The system innovatively combines LiDAR point cloud depth information with the geometric constraints of visual line features and introduces a PLC descriptor specifically designed for large-scale outdoor scenes, thereby improving loop closure detection. Experimental results demonstrate that PL-LVI outperforms state-of-the-art systems in both localization accuracy and robustness. Furthermore, the incorporation of line features mitigates the stability limitations of traditional point-based methods in structured environments, offering a promising approach to addressing sensor degradation issues.

Looking ahead, we plan to integrate plane constraints into PLC to improve adaptability in indoor environments. Additionally, future research will focus on optimizing computational efficiency and exploring synergies with deep learning to enhance the performance of PL-LVI in partially or fully degraded conditions.

REFERENCES

[1] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse, "MonoSLAM: Real-time single camera SLAM," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 6, pp. 1052–1067, Jun. 2007.

[2] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[3] J. Zhang and S. Singh, "LOAM: LiDAR odometry and mapping in real-time," *Robot., Sci. Syst.*, vol. 2, no. 9, pp. 1–9, 2014.

[4] K. Liu and M. Cao, "DLC-SLAM: A robust LiDAR-SLAM system with learning-based denoising and loop closure," *IEEE/ASME Trans. Mechatronics*, vol. 28, no. 5, pp. 2876–2884, Oct. 2023.

[5] X. Zuo, P. Geneva, W. Lee, Y. Liu, and G. Huang, "LIC-Fusion: LiDAR-inertial-camera odometry," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 5848–5854.

[6] T. Shan, B. Englot, C. Ratti, and D. Rus, "LVI-SAM: Tightly-coupled LiDAR-visual-inertial odometry via smoothing and mapping," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 5692–5698.

[7] J. Lin and F. Zhang, "R^3LIVE++: A robust, real-time, radiance reconstruction package with a tightly-coupled LiDAR-inertial-visual state estimator," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 11168–11185, Dec. 2024.

[8] G. P. C. Junior et al., "EKF-LOAM: An adaptive fusion of LiDAR SLAM with wheel odometry and inertial data for confined spaces with few geometric features," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 1458–1471, Jul. 2022.

[9] Z. Liu, D. Shi, R. Li, W. Qin, Y. Zhang, and X. Ren, "PLC-VIO: Visual–inertial odometry based on point-line constraints," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 3, pp. 1880–1897, Jul. 2022.

[10] H. Liang, N. J. Sanket, C. Fermuller, and Y. Aloimonos, "SalientDSO: Bringing attention to direct sparse odometry," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 4, pp. 1619–1626, Oct. 2019.

[11] Y. He, J. Zhao, Y. Guo, W. He, and K. Yuan, "PL-VIO: Tightly-coupled monocular visual–inertial odometry using point and line features," *Sensors*, vol. 18, no. 4, p. 1159, Apr. 2018.

[12] Q. Fu et al., "PL-VINS: Real-time monocular visual-inertial SLAM with point and line features," 2020, *arXiv:2009.07462*.

[13] G. Yang, Q. Wang, P. Liu, and C. Yan, "PLS-VINS: Visual inertial state estimator with point-line features fusion and structural constraints," *IEEE Sensors J.*, vol. 21, no. 24, pp. 27967–27981, Dec. 2021.

[14] R. G. von Gioi, J. Jakubowicz, J.-M. Morel, and G. Randall, "LSD: A fast line segment detector with a false detection control," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 4, pp. 722–732, Apr. 2010.

[15] D. Li et al., "A SLAM system based on RGBD image and point-line feature," *IEEE Access*, vol. 9, pp. 9012–9025, 2021.

[16] W. Shao, S. Vijayarangan, C. Li, and G. Kantor, "Stereo visual inertial LiDAR simultaneous localization and mapping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 370–377.

[17] Z. Wang, J. Zhang, S. Chen, C. Yuan, J. Zhang, and J. Zhang, "Robust high accuracy visual-inertial-laser SLAM system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Nov. 2019, pp. 6636–6641.

[18] J. Zhang and S. Singh, "Laser–visual–inertial odometry and mapping with high robustness and low drift," *J. Field Robot.*, vol. 35, no. 8, pp. 1242–1264, Dec. 2018.

[19] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint Kalman filter for vision-aided inertial navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, Apr. 2007, pp. 3565–3572.

[20] X. Zuo et al., "LIC-fusion 2.0: LiDAR-inertial-camera odometry with sliding-window plane-feature tracking," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 5112–5119.

[21] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "LIO-SAM: Tightly-coupled LiDAR inertial odometry via smoothing and mapping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2020, pp. 5135–5142.

[22] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.

[23] A. Pumarola, A. Vakhitov, A. Agudo, A. Sanfeliu, and F. Moreno-Noguer, "PL-SLAM: Real-time monocular visual SLAM with points and lines," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, May 2017, pp. 4503–4508.

[24] G. Ren, Z. Cao, X. Liu, M. Tan, and J. Yu, "PLJ-SLAM: Monocular visual SLAM with points, lines, and junctions of coplanar lines," *IEEE Sensors J.*, vol. 22, no. 15, pp. 15465–15476, Aug. 2022.

[25] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An open-source SLAM system for monocular, stereo, and RGB-D cameras," *IEEE Trans. Robot.*, vol. 33, no. 5, pp. 1255–1262, Oct. 2017.

[26] H. Jiang, R. Qian, L. Du, J. Pu, and J. Feng, "UL-SLAM: A universal monocular line-based SLAM via unifying structural and non-structural constraints," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 2682–2699, 2025.

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.

[27] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
[28] W. Guan, P. Chen, Y. Xie, and P. Lu, "PL-EVIO: Robust monocular event-based visual inertial odometry with point and line features," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 6277–6293, Oct. 2024.
[29] W. Guan and P. Lu, "Monocular event visual inertial odometry based on event-corner using sliding windows graph-based optimization," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2022, pp. 2438–2445.
[30] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Aug. 1994, pp. 593–600.
[31] S. Baker and I. Matthews, "Lucas–Kanade 20 years on: A unifying framework," *Int. J. Comput. Vis.*, vol. 56, no. 3, pp. 221–255, 2004.
[32] L. Zhang and R. Koch, "An efficient and robust line segment matching approach based on LBD descriptor and pairwise geometric consistency," *J. Vis. Commun. Image Represent.*, vol. 24, no. 7, pp. 794–805, Oct. 2013.
[33] A. Zelinsky, "Learning OpenCV—Computer vision with the OpenCV library," *IEEE Robot. Autom. Mag.*, vol. 16, no. 3, p. 100, Sep. 2009.
[34] A. Bartoli and P. Sturm, "Structure-from-motion using lines: Representation, triangulation, and bundle adjustment," *Comput. Vis. Image Understand.*, vol. 100, no. 3, pp. 416–441, Dec. 2005.
[35] A. Bartoli and P. Sturm, "The 3D line motion matrix and alignment of line reconstructions," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 1, Dec. 2001, pp. 1–45.
[36] G. Kim and A. Kim, "Scan context: Egocentric spatial descriptor for place recognition within 3D point cloud map," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 4802–4809.
[37] L.-T. Hsu et al., "UrbanNav: An open-sourced multisensory dataset for benchmarking positioning algorithms designed for urban areas," in *Proc. 34th Int. Tech. Meeting Satell. Division Inst. Navigat.*, Oct. 2021, pp. 226–256.
[38] J. Yin, A. Li, T. Li, W. Yu, and D. Zou, "M2DGR: A multi-sensor and multi-scenario SLAM dataset for ground robots," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2266–2273, Apr. 2022.
[39] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized LiDAR odometry and mapping on variable terrain," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2018, pp. 4758–4765.
[40] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2053–2073, Aug. 2022.
[41] D. Galvez-López and J. D. Tardos, "Bags of binary words for fast place recognition in image sequences," *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1188–1197, Oct. 2012.
[42] M. Quigley, "ROS: An open-source robot operating system," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Jan. 2009, p. 5.
[43] T. Shan, B. Englot, F. Duarte, C. Ratti, and D. Rus, "Robust place recognition using an imaging LiDAR," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 5469–5475.
[44] Q. Sun, J. Yuan, X. Zhang, and F. Duan, "Plane-edge-SLAM: Seamless fusion of planes and edges for SLAM in indoor environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 4, pp. 2061–2075, Oct. 2021.



Feng Hui received the B.S. degree in automation from Southwest Petroleum University, Chengdu, China, in 2020, and the M.Eng. degree in automatic control from South China University of Technology, Guangzhou, China, in 2024, where he is currently pursuing the Ph.D. degree.

His current research interests include robotics, computer vision, and simultaneous localization and mapping (SLAM).



Zhiyu Zhou (Student Member, IEEE) received the M.Eng. degree in automatic control from South China University of Technology, Guangzhou, China, in 2023, where he is currently pursuing the D.Eng. degree in electronic and information engineering with the School of Automation Science and Engineering.

His research interests include 3-D vision, neural rendering, and robotic perception.



Yu Liu (Senior Member, IEEE) received the Ph.D. degree in automatic control from South China University of Technology, Guangzhou, China, in 2009.

He was a Visiting Student with the Department of Mechanical Engineering, Concordia University, Montreal, QC, Canada, from 2008 to 2009, and a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Nebraska-Lincoln, Lincoln, NE, USA, from 2017 to 2018. He is currently a Professor with the School of Automation Science and Engineering, South China University of Technology, and the Research and Development Center of Precision Electronic Manufacturing Technology, Guangzhou Institute of Modern Industrial Technology, Guangzhou. His current research interests include robotics, intelligent control systems, and machine vision.

Dr. Liu is currently an Associate Editor of IEEE/ASME TRANSACTIONS ON MECHATRONICS, IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, IEEE TRANSACTIONS ON FUZZY SYSTEMS, and IEEE TRANSACTIONS ON COMPUTATIONAL SOCIAL SYSTEMS.

Authorized licensed use limited to: Soochow University. Downloaded on September 15,2026 at 08:08:46 UTC from IEEE Xplore. Restrictions apply.